

MEDIROZA GENERAL HOSPITAL

INDEPENDENT WEB APPLICATION SECURITY

ASSESSMENT

PORTFOLIO-SAFE CASE STUDY

Prepared by Chandrashekar Bala

Authorized black-box assessment • 01 October 2026 • Public-safe edition

Credentials, patient records, national identifiers, direct contact data and raw sensitive artifacts are intentionally excluded.

Project Overview

This standalone case study demonstrates an end-to-end web application security assessment performed from an external, unauthenticated position under written authorization. The work combines reconnaissance, manual exploitation, evidence validation, document-security analysis, impact quantification, remediation engineering and retest design.

Capability	Demonstrated work
Web reconnaissance	DNS, HTTP metadata, robots/sitemap, application mapping
Offensive testing	Authentication testing, SQL injection, directory exposure, object-access analysis
Tooling	Nmap, curl, Burp Suite, sqlmap, qpdf, pdftcrack
Evidence engineering	Baseline/exploit comparison, screenshots, hashes, command register
Defensive engineering	Root-cause analysis, remediation patterns, detection rules, regression tests

Executive Findings

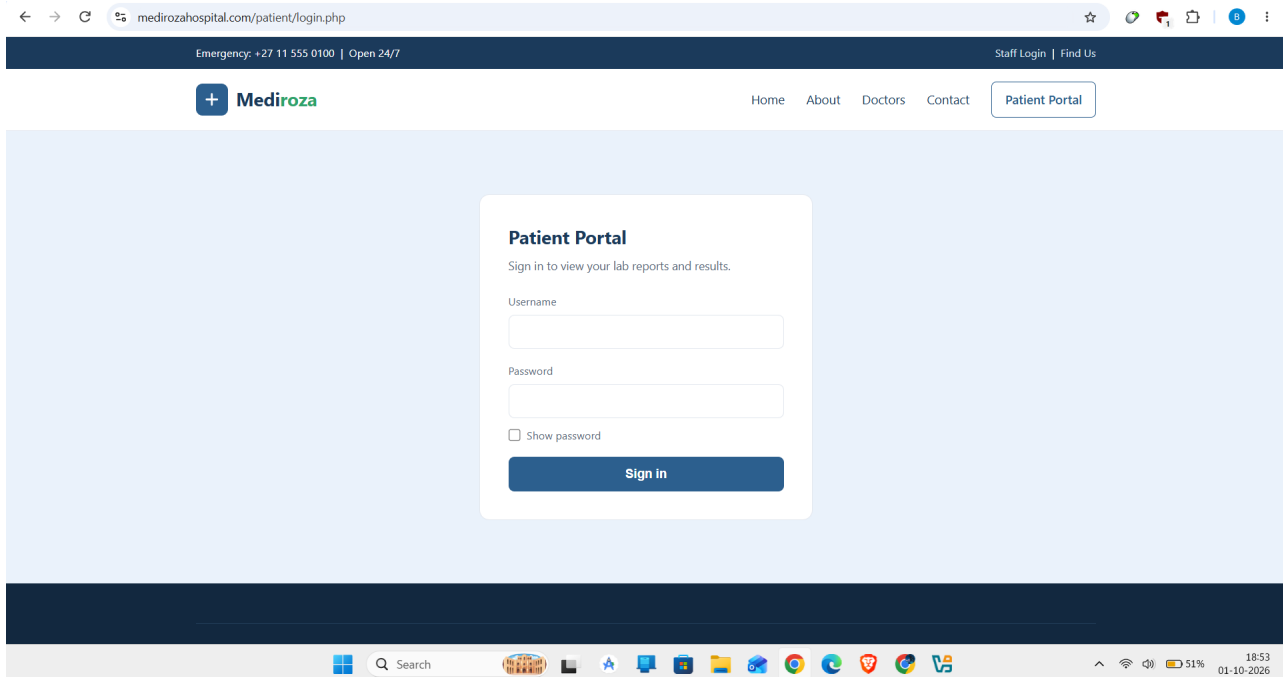
ID	Finding	Severity	CVSS
F-01	Unauthenticated directory indexing	HIGH	7.5
F-02	Public internal database backup exposure	CRITICAL	9.4*
F-03	SQL injection authentication bypass	CRITICAL	9.8
F-04	Verbose SQL error disclosure	MEDIUM	5.3
F-05	Sensitive report retrieval after auth bypass	HIGH	8.1*
F-06	Predictable/disclosed report path	LOW	3.1*
F-07	Error-log metadata disclosure	MEDIUM	5.3*
F-08	Server/hosting fingerprint disclosure	LOW	3.1*

The strongest technical result is the reproducible authentication-state change: a normal invalid login returned HTTP 200 with no redirect, while a controlled SQL syntax mutation returned HTTP 302 to the patient portal. Browser evidence confirmed the resulting portal state.

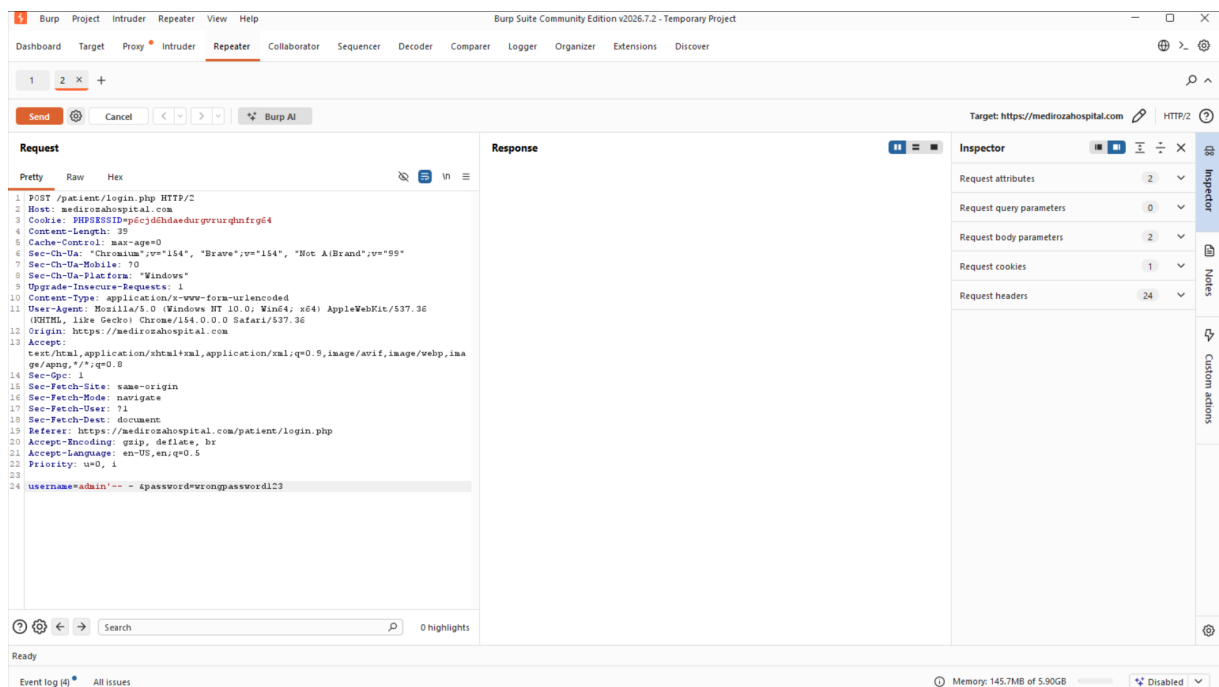
Technical Deep Dive — SQL Injection

The login endpoint accepted username and password via POST. The baseline request established normal behavior. The exploit request changed only the username field. The exact server-side source code was not available, so the root cause is described at the observable control level: untrusted input is influencing database query behavior.

A secure implementation should use parameterized queries so the database receives the username as data rather than executable SQL syntax. Production error handling should also suppress database-driver messages and preserve diagnostics only in protected server-side logs.



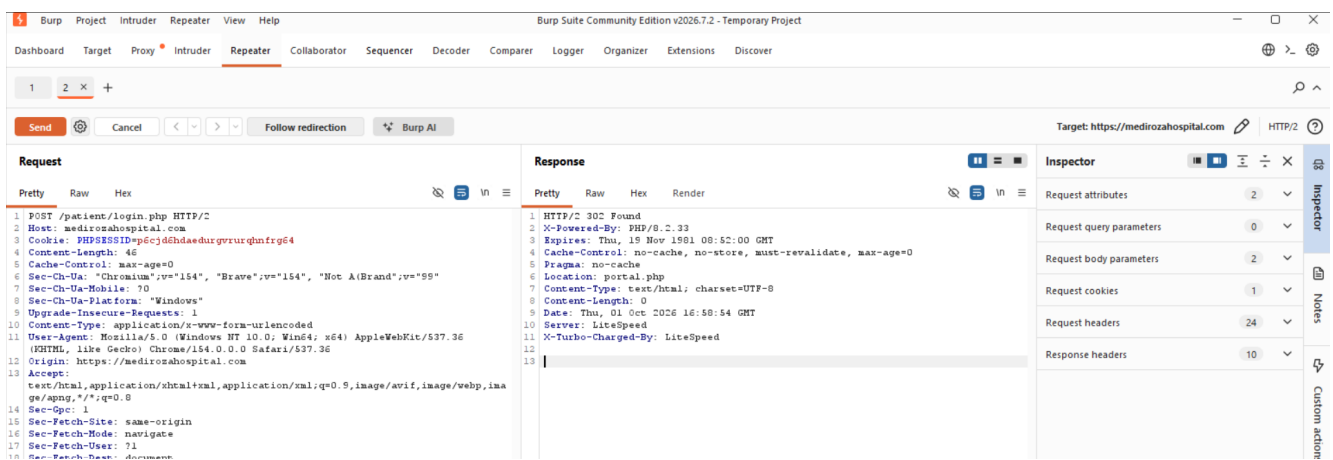
Patient login surface — public-safe evidence.



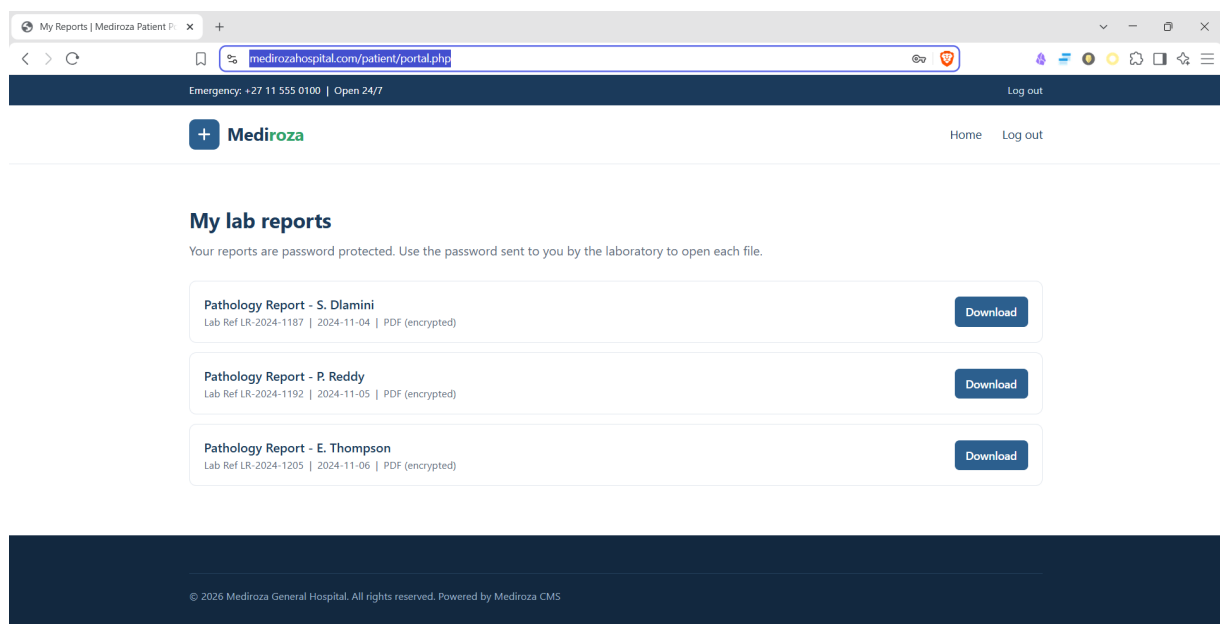
Controlled SQL injection test — public-safe evidence.

Evidence Quality & Reproducibility

A strong vulnerability report includes a baseline, a minimal exploit, corroborating evidence, honest tool limitations and an explicit impact demonstration.



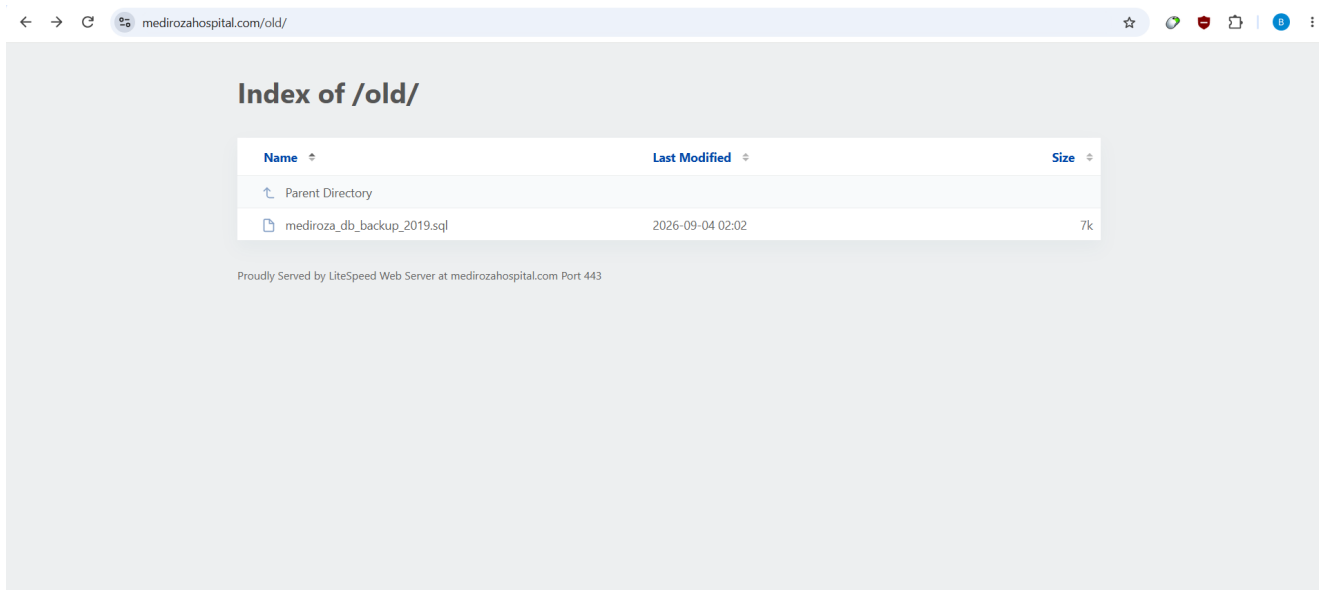
HTTP 302 authentication-bypass response.



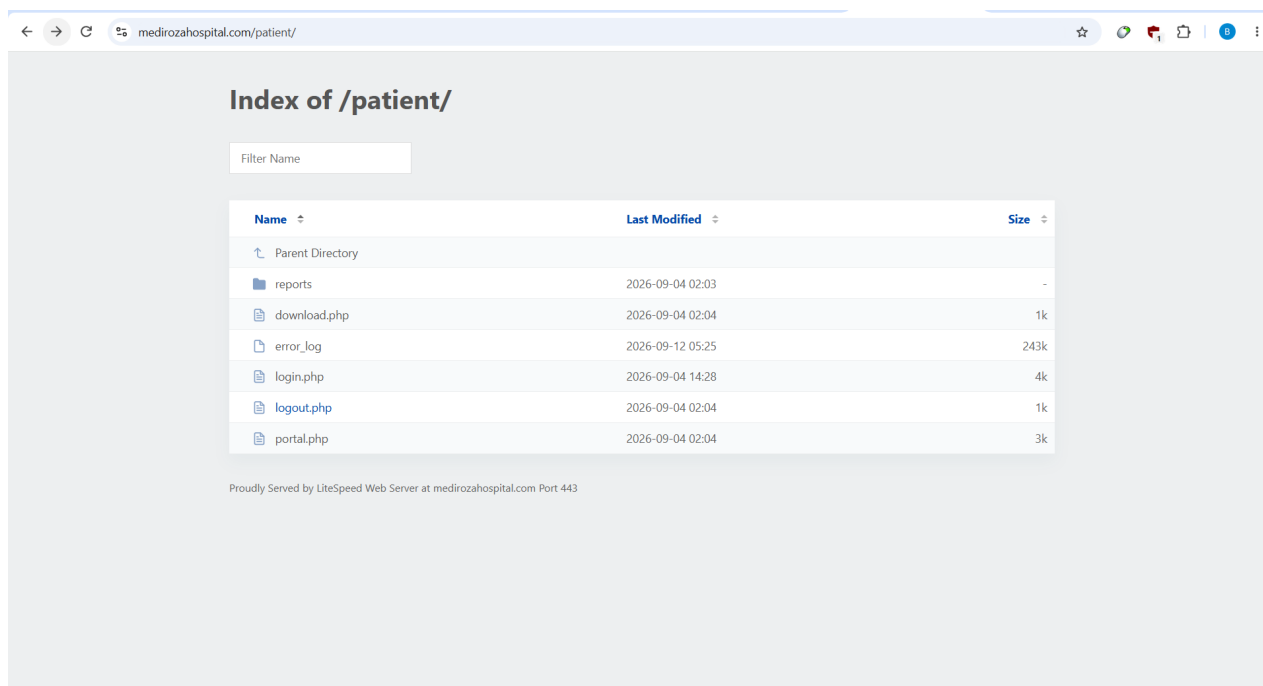
Browser-confirmed portal access.

Directory & Backup Exposure

Directory indexing exposed application structure under restricted paths. The most consequential result was discovery of a database backup in a legacy directory. The public-safe version does not reproduce the sensitive database contents; the confidential edition contains the complete technical record.



Legacy directory listing — public-safe context.



Patient directory enumeration evidence.

Document Security Analysis

Three encrypted PDF artifacts were retrieved after the authentication boundary was bypassed. The assessment recorded PDF 1.4, Standard security-handler parameters V=2/R=3 and 128-bit protection. Authorized offline password recovery succeeded against all three files. Passwords and report contents are excluded from this public-safe version.

```
(kali@kali): ~/Weeks/Mediroza/evidence/pdf
$ ls
patient_report_1.hash  patient_report_1.pdf  patient_report_2.pdf  patient_report_3.pdf

(kali@kali): ~/Weeks/Mediroza/evidence/pdf
$ cat patient_report_1.pdf > patient_report_1.hash

(kali@kali): ~/Weeks/Mediroza/evidence/pdf
$ cat patient_report_2.pdf > patient_report_2.hash

(kali@kali): ~/Weeks/Mediroza/evidence/pdf
$ cat patient_report_3.pdf > patient_report_3.hash

(kali@kali): ~/Weeks/Mediroza/evidence/pdf
$ cat patient_report_1.hash
cat: patient_report_1.hash: No such file or directory

(kali@kali): ~/Weeks/Mediroza/evidence/pdf
$ cat patient_report_1.hash
patient_report_1.pdf:pdf2*3*128*4294967292*1*32*316636336326235641333353161323830313731623833323837376335339*32*ef16c5d
ad8fce118c79e9d28895b52bf4e5ee758aa164084e56ffafad108*32*c431fab9cc5e77b59c244861d745771ac5ba427b19192da468e77127f1e9d6

(kali@kali): ~/Weeks/Mediroza/evidence/pdf
$ cat patient_report_2.hash
patient_report_2.pdf:pdf2*3*128*4294967292*1*32*31663633637236356a376264436383466373730326563666363264616463*32*39f4e6b
8aedf2244c240ffbf9f08932b0f4e5ee758aa164084e56ffafad108*32*c431fab9cc5e77b59c244861d745771ac5ba427b19192da468e77127f1e9d6

(kali@kali): ~/Weeks/Mediroza/evidence/pdf
$ cat patient_report_3.hash
patient_report_3.pdf:pdf2*3*128*4294967292*1*32*326139386326138336343863772361186164372264323138161376165168*32*5090f8a
d5b899379fc9d146c23119a28f4e5ee758aa164084e56ffafad108*32*c431fab9cc5e77b59c244861d745771ac5ba427b19192da468e77127f1e9d6

(kali@kali): ~/Weeks/Mediroza/evidence/pdf
```

PDF hash extraction — sensitive values excluded from public release.

```
(kali@kali): ~/Weeks/Mediroza/evidence/pdf
$ pdftocrack --help
pdftocrack: unrecognized option '--help'
Usage: pdftocrack -f filename [OPTIONS]

OPTIONS:
  -b, --bench           perform benchmark and exit
  -c, --charset=STRING  Use the characters in STRING as charset
  -w, --wordlist=FILE    Use FILE as source of passwords to try
  -m, --minpw=INTEGER   Skip trying passwords shorter than this
  -M, --maxpw=INTEGER   Stop when reaching this passwordlength
  -l, --loadstate=FILE  Continue from the state saved in FILENAME
  -o, --owner           Work with the ownerpassword
  -u, --user            Work with the userpassword (default)
  -p, --password=STRING Give userpassword to speed up breaking
                        ownerpassword (implies -o)
  -q, --quiet          Run quietly
  -s, --permutate       Try permutating the passwords (currently only
                        supports switching first character to uppercase)
  -v, --version         Print version and exit

(kali@kali): ~/Weeks/Mediroza/evidence/pdf
$ pdftocrack --version
pdftocrack: unrecognized option '--version'
Usage: pdftocrack -f filename [OPTIONS]

OPTIONS:
  -b, --bench           perform benchmark and exit
  -c, --charset=STRING  Use the characters in STRING as charset
  -w, --wordlist=FILE    Use FILE as source of passwords to try
  -m, --minpw=INTEGER   Skip trying passwords shorter than this
  -M, --maxpw=INTEGER   Stop when reaching this passwordlength
  -l, --loadstate=FILE  Continue from the state saved in FILENAME
  -o, --owner           Work with the ownerpassword
  -u, --user            Work with the userpassword (default)
  -p, --password=STRING Give userpassword to speed up breaking
                        ownerpassword (implies -o)
  -q, --quiet          Run quietly
  -s, --permutate       Try permutating the passwords (currently only
                        supports switching first character to uppercase)
  -v, --version         Print version and exit

(kali@kali): ~/Weeks/Mediroza/evidence/pdf
$ which pdftocrack
/usr/bin/pdftocrack

(kali@kali): ~/Weeks/Mediroza/evidence/pdf
$ ls
patient_report_1.hash  patient_report_1_john.hash  patient_report_1.pdf  patient_report_2.hash  patient_report_2.pdf  patient_report_3.hash  patient_report_3.pdf

(kali@kali): ~/Weeks/Mediroza/evidence/pdf
$ pdftocrack -u /usr/share/wordlists/rockyou.txt patient_report_1.pdf > patient_report_1_cracked.txt

(kali@kali): ~/Weeks/Mediroza/evidence/pdf
$ ls
patient_report_1_cracked.txt  patient_report_1.hash  patient_report_1_john.hash  patient_report_1.pdf  patient_report_2.hash  patient_report_2.pdf  patient_report_3.hash  patient_report_3.pdf

(kali@kali): ~/Weeks/Mediroza/evidence/pdf
$ cat patient_report_1_cracked.txt
PDF version 1.4
Security Handler: Standard
V: 2
R: 3
P: 4
Length: 128
Encrypted Metadata: True
```

Authorized document-security testing.

Impact Quantification

The confidential database backup contained structured staff and shareholder datasets. The technical analysis established 30 staff records and 10 shareholder records and calculated an exact monthly payroll total of ZAR 2,027,000. Individual identities and raw rows are excluded from this public version.

Data class	Observed quantity
Staff records	30
Shareholder records	10
Monthly payroll total	ZAR 2,027,000
Patient report artifacts	3 retrieved PDFs

Root Causes

- Sensitive artifacts were placed under the public web document root.
- Directory indexing disclosed files that should have been private.
- Authentication input was not safely isolated from database query semantics.
- Sensitive report objects were reachable after the authentication boundary was bypassed.
- Production errors disclosed database implementation details.

Remediation Architecture

Control	Engineering action
Public-file hygiene	Move backups/logs/archives outside the web root
Authentication	Use prepared statements and least-privileged DB access
Object authorization	Authorize every document request against session identity and record ownership
Error handling	Return generic client errors; log details server-side
Monitoring	Correlate failed login sequences with portal-session creation
CI/CD	Block database dumps and backup artifacts from deployment

Retesting should verify the security invariant rather than only one payload. Directory indexes should be gone; known backup names should return 403/404; SQL-like inputs should never change authentication state; unauthorized report IDs should return a consistent denial; and expected security telemetry should be visible to defenders.

Resume-ready Project Description

Independent Web Application Security Assessment — Conducted an authorized black-box assessment of a hospital web application; identified and manually validated SQL-injection authentication bypass, public database-backup exposure, directory indexing and sensitive document retrieval; analyzed legacy PDF protection; quantified exposed corporate data; and produced evidence-backed severity, remediation, detection and retest artifacts.

Public Release Boundary

Use this sanitized edition for a portfolio, GitHub repository or resume-linked project page. Keep the confidential technical report and evidence package private because they contain recovered credentials and sensitive personal/corporate data. The public version preserves the technical story without exposing those records.